

Atividade 04 - Heapsort

Implemente, preferencialmente em C++, o método de ordenação Heapsort, usando-se como base o material usado nas aulas. Monte um arquivo PDF contando resumidamente como fez a atividade, quais problemas teve e como os superou, colocando prints do código e do resultado para ilustrar o que foi construído.

Relato: Fiz a atividade com o auxílio do material da aula, fiz algumas substituições e criei mais algumas coisas. Assim como, realizei a troca do vetor do tipo struct do material por um vetor de caracteres. Não tive problemas ao fazer essa atividade, consegui entender muito bem o conceito e funcionamento devido à explicação do professor.

CÓDIGO:

```
#include <iostream>
#include <vector>

using namespace std;

void troca(char *a, char *b)
{
    char aux;
    aux = *a;
    *a = *b;
    *b = aux;
}

void refaz_cima_baixo(vector<char> &carac, int k, int N)
{
    int j;
    while (2 * k <= N)
    {
        j = 2 * k;
        if (j < N && carac[j] < carac[j + 1])
        {
            j++;
        }
    }
}
```

```

    }
    if (carac[k] >= carac[j])
    {
        break;
    }
    troca(&carac[k], &carac[j]);
    k = j;
}
}

void refaz_baixo_cima(vector<char> &carac, int k)
{
    while (k > 1 && carac[k / 2] < carac[k])
    {
        troca(&carac[k], &carac[k / 2]);
        k = k / 2;
    }
}

void heap_constroi(vector<char> &carac, int n)
{
    int k = n / 2;
    while (k >= 1)
    {
        refaz_cima_baixo(carac, k, n);
        k--;
    }
}

// Insere um novo item no heap
void heap_insere(vector<char> &carac, int *n, char novo)
{
    carac.push_back(novo);
    *n = carac.size() - 1;
    refaz_baixo_cima(carac, *n);
}

// Remove o maior item do heap (a raiz)
char heap_remove_maximo(vector<char> &carac, int *n)
{
    char maximo = carac[1];
    troca(&carac[1], &carac[*n]);

```

```

        *n -= 1;
        carac.pop_back();
        if (*n > 0)
        {
            refaz_cima_baixo(carac, 1, *n);
        }
        return maximo;
    }

void heapsort(vector<char> &carac, int n)
{
    heap_constroi(carac, n);
    int k = n;
    while (k >= 1)
    {
        troca(&carac[1], &carac[k]);
        k--;
        refaz_cima_baixo(carac, 1, k);
    }
}

void imprime_vetor(const vector<char> &carac, int n)
{
    for (int i = 1; i <= n; i++)
    {
        cout << carac[i] << " ";
    }
    cout << endl;
}

int main()
{
    vector<char> caracteres = {' ', 'E', 'Y', 'O', 'A', 'C', 'F', 'P',
'V'};
    int n = caracteres.size() - 1;

    cout << "-----\n";
    cout << "Vetor original: ";
    imprime_vetor(caracteres, n);
    cout << "-----\n";

    cout << "1. Construindo o heap (heap_constroi)" << endl;
}

```

```

    heap_constroi(caracteres, n);
    cout << "Vetor apos virar um heap: ";
    imprime_vetor(caracteres, n);
    cout << "-----\n";

    cout << "2. Inserindo o caractere 'Z' (heap_insere)" << endl;
    heap_insere(caracteres, &n, 'Z');
    cout << "Heap apos inserir 'Z': ";
    imprime_vetor(caracteres, n);
    cout << "-----\n";

    cout << "3. Removendo o elemento maximo (heap_remove_maximo)" <<
endl;
    cout << "Elemento maximo removido: " <<
heap_remove_maximo(caracteres, &n) << endl;
    cout << "Heap apos remover o maximo: ";
    imprime_vetor(caracteres, n);
    cout << "-----\n";

    cout << "4. Ordenando o vetor com heapsort" << endl;
    vector<char> vetor_para_ordenar = {' ', 'E', 'Y', 'O', 'A', 'C',
'F', 'P', 'V'};
    int n_ordenar = vetor_para_ordenar.size() - 1;
    cout << "Vetor antes do heapsort: ";
    imprime_vetor(vetor_para_ordenar, n_ordenar);

    heapsort(vetor_para_ordenar, n_ordenar);

    cout << "Vetor ordenado: ";
    // Após o heapsort, os elementos estão nas posições 1 a n_ordenar
    imprime_vetor(vetor_para_ordenar, n_ordenar);
    cout << "-----\n";

    return 0;
}

```

RESULTADO:

```
-----  
Vetor original: E Y O A C F P V  
-----
```

```
1. Construindo o heap (heap_constroi)
```

```
Vetor apos virar um heap: Y V P E C F O A  
-----
```

```
2. Inserindo o caractere 'Z' (heap_inserir)
```

```
Heap apos inserir 'Z': Z Y P V C F O A E  
-----
```

```
3. Removendo o elemento maximo (heap_remove_maximo)
```

```
Elemento maximo removido: Z
```

```
Heap apos remover o maximo: Y V P E C F O A  
-----
```

```
4. Ordenando o vetor com heapsort
```

```
Vetor antes do heapsort: E Y O A C F P V
```

```
Vetor ordenado: A C E F O P V Y  
-----
```